Riel Kediende Mapuor Akec

Electrical & Electronics Engineering graduate, Universiti Teknologi PETRONAS

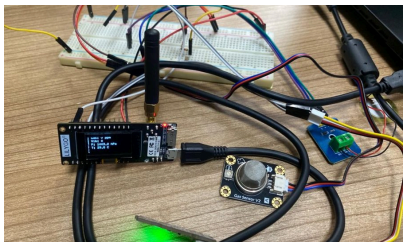
riel.akec@alum.utp.edu.my
linkedin.com/in/rielkediende

LoRa-Based Point-to-Point Leak Detection System for Remote Pipelines

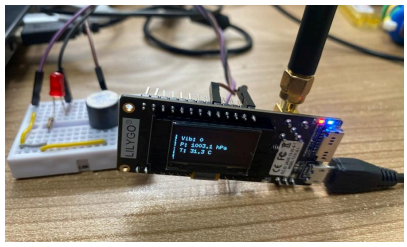
Final Year Project, BEng (Hons) Electrical & Electronics Engineering, 2025 | Hardware, firmware and field testing by Riel

A two-node wireless system that detects gas leaks and mechanical disturbances on remote pipelines without gateways or internet. The transmitter (ESP32 + SX1276, LilyGO LoRa32 V2.1) samples a methane sensor (MQ-4, ADC), a barometric pressure and temperature sensor (DPS310, I²C) and a piezoelectric vibration sensor (ADC), averages and packs the readings into a compact payload, and sends it at 923 MHz in the Malaysian ISM band. The receiver decodes each packet, shows live values on an OLED and triggers an LED and buzzer when thresholds are crossed.

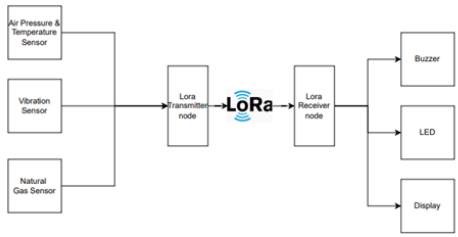
500 m	100%	0	75–1,083 ms
outdoor link tested, semi-urban	detection accuracy up to 300 m	false alarms in 250 trials	latency, SF7 to SF12



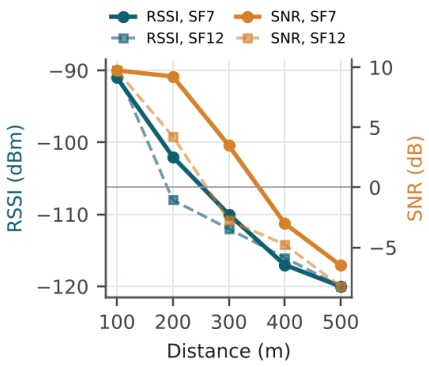
Transmitter node. ESP32/SX1276 with MQ-4 gas, DPS310 pressure/temperature and piezo vibration sensors.



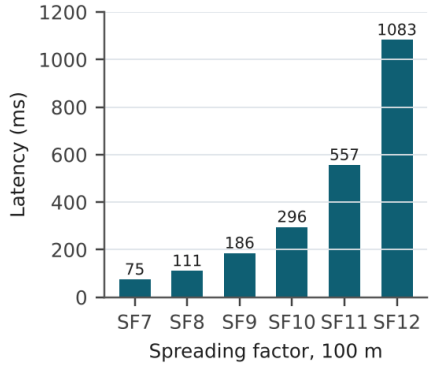
Receiver node. Live readings on OLED (pressure 1002.1 hPa, 31.3 °C shown) with LED and buzzer alarm.



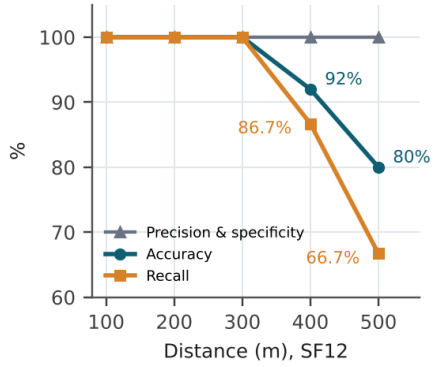
System architecture. Three sensors feed the transmitter; the receiver drives buzzer, LED and display.



Link quality vs distance. At SF7, RSSI fell from −91 to −120 dBm and SNR from +9.8 to −6.5 dB; SF12 still decoded at −8.25 dB SNR.



Spreading-factor trade-off. Latency rose from 75 ms (SF7) to 1,083 ms (SF12), matching LoRa time-on-air theory.



Leak detection reliability. 50 trials per distance (30 leak, 20 no-leak). Perfect to 300 m; no false alarms at any range.

What I did

- Selected components and drew transmitter and receiver schematics (GPIO, ADC and I²C interfacing).
- Wrote Arduino/C++ firmware for sampling, averaging, payload encoding, LoRa TX/RX and threshold alerts.
- Planned and ran a three-phase field test: distance sweep, SF7–SF12 sweep and a 250-trial confusion-matrix evaluation.
- Computed accuracy, precision, recall and specificity, and analysed results against LoRa link-budget theory.

All figures and measurements are from the author's FYP dissertation, Universiti Teknologi PETRONAS. Charts redrawn from the dissertation's result tables.

Riel Kediende Mapuor Akec

Electrical & Electronics Engineering graduate, Universiti Teknologi PETRONAS

riel.akec@alum.utp.edu.my
linkedin.com/in/rielkediende

IoT Service Module: Autonomous Shape-Guided Robot

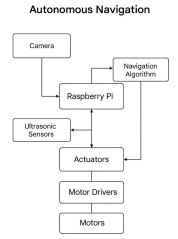
Capstone, Integrated System Design Project (EEB4022), UTP, 2025 | My subsystems: Image Transmission & Processing and Autonomous Navigation

A Raspberry Pi 5 robot that travels to a remote plant whose temperature sensor has failed, finding its way by recognising triangle, circle and square markers, then takes over the plant's cooling-fan control from a base-station laptop. The team split the system into four subsystems; I owned two of them: the vision pipeline that sees and streams, and the navigation logic that turns what it sees into motor commands.

2 of 4	3 shapes	120°	Live
subsystems owned in a 3-person team	custom YOLO classes: triangle, circle, square	field of view, Pi Camera Module 3	video to base station via Viam cloud



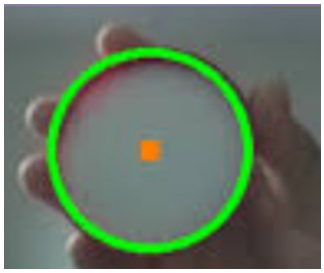
Vision pipeline. Camera to Raspberry Pi 5 (YOLO inference) to base-station laptop.



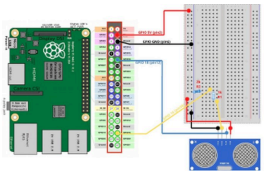
Navigation architecture. Camera and ultrasonic inputs feed the navigation algorithm, which drives the motors.



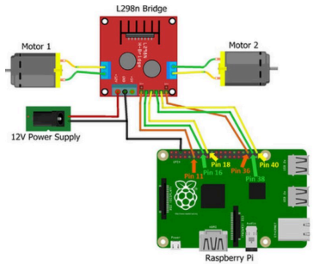
On-device detection. YOLO labelling objects with confidence scores (faces blurred).



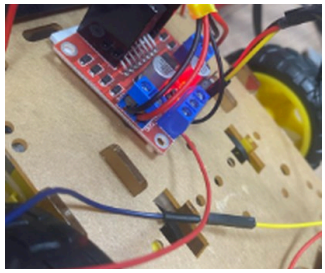
Shape target. Circle marker detected and centred for approach.



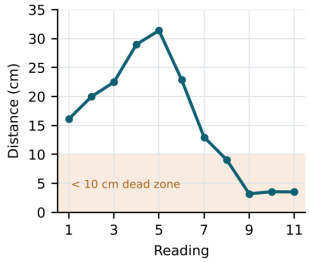
Ultrasonic wiring to Raspberry Pi 5 GPIO for obstacle ranging.



Drive wiring. L298N H-bridge driving two TT gear motors with PWM from the Pi.



Chassis build with motor driver and wiring.



Live ultrasonic readings logged during testing, 3.2 to 31.4 cm.

Design decisions I made

- **YOLO over OpenCV contours:** classifies custom shapes reliably despite rotation and partial occlusion, and still runs in near real time on the Pi 5 with a nano model.
- **Viam over Blynk and Arduino Cloud:** the only option with native low-latency video streaming and image storage on its free tier.
- **YOLO + ultrasonic fusion:** the camera finds the target; ultrasonic ranging stops, pivots or slows the robot near obstacles, and works in low light.

What testing showed

- Shape detection held up with slight occlusion and rotation; pretrained weights also generalised to everyday objects.
- Accuracy dropped in poor light and clutter. Proposed fixes: IR lighting, lower input resolution, or an edge TPU for inference.
- Ultrasonic ranging was unreliable under 10 cm and on glass or fabric, so camera depth or IR sensing should back it up.

Figures from the team's ISDP I Critical Design Review report (January 2025), Universiti Teknologi PETRONAS. Readings chart redrawn from the logged sensor output.

Riel Kediende Mapuor Akec

Electrical & Electronics Engineering graduate, Universiti Teknologi PETRONAS

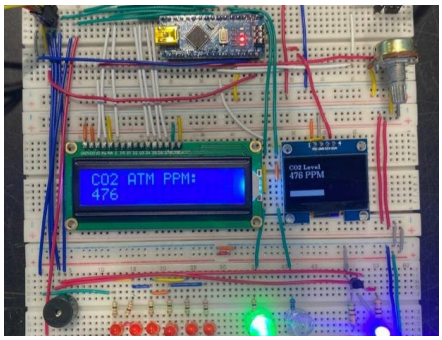
riel.akec@alum.utp.edu.my
linkedin.com/in/rielkediende

Direct Air Capture Prototype: CO_2 Monitoring and Control Electronics

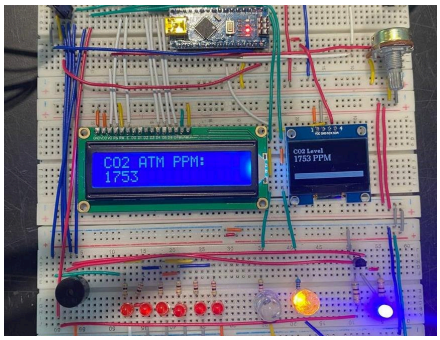
Engineering Team Project (MEB2063), UTP, September 2023 | Assistant Project Director; sole electrical engineer in a 7-member team (chemical, mechanical, petroleum, electrical)

A tabletop chamber that pulls ambient air across a 5% potassium hydroxide solution to absorb CO_2. I designed and programmed the electronics: an Arduino Nano reads an MQ135 gas sensor, averages 10 samples, converts the result to ppm and shows it live on an OLED (with a bar graph) and a 16×2 LCD. Three LED levels and a 3 kHz buzzer warn at rising CO_2, and a transistor switch runs the intake fan and the solution pump from a 7.4 V battery.

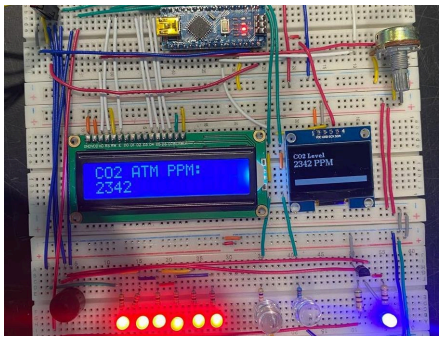
45%	35%	3 levels	RM269
CO_2 reduction, closed test (840 → 462 ppm)	reduction, open-air test (611 → 396 ppm)	alarm: ≤1,000 / 1,001–1,999 / ≥2,000 ppm	total build cost



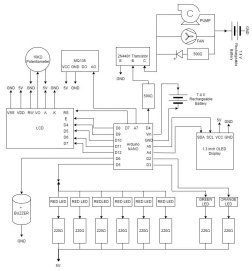
476 ppm: green. Safe level; LCD and OLED show the live reading.



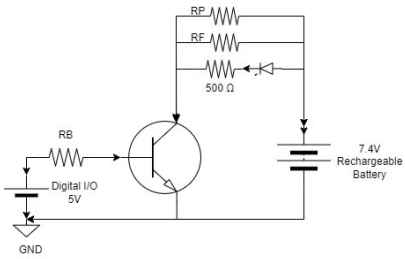
1,753 ppm: orange. Elevated level warning.



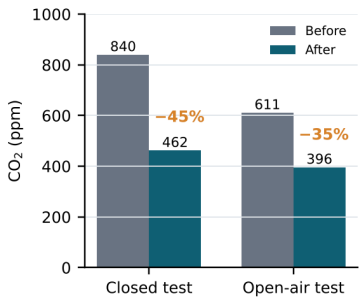
2,342 ppm: red + buzzer. Dangerous level; red LED bank and 3 kHz tone.



Full circuit schematic: Nano, MQ135, OLED (I^2C), LCD, LED bank, buzzer and fan/pump driver.



Fan and pump driver. Load and base-current calculation: 600 mA pump + 340 mA fan, base current 8.6 mA to stay under the Nano's pin limit.



Measured results. CO_2 before and after running the prototype, read by the monitoring circuit.

Sensor selection

	MQ135	MG811	MH-Z19B
Price	**RM7.50**	RM123.87	RM42.93
Preheat	**20 s**	3 min	3 min
Range	100–1,000 ppm	350–10,000	0–10,000
Gases	CO_2 + others	CO_2	CO_2

Chose the MQ135 for cost and fast warm-up. It responds to several gases, so a dedicated NDIR sensor (MH-Z19B) is the upgrade for absolute CO_2 accuracy.

What I did

- Drew the circuit and driver schematics and wrote the C++ firmware (sampling, averaging, ppm mapping, alarms, dual display).
- Sized the fan and pump switch from load, base-current and forced-beta calculations.
- As assistant project director, set weekly agendas, assigned tasks and tracked progress across four disciplines.

Photos, schematics and measurements from the team's Engineering Team Project report, Universiti Teknologi PETRONAS. Results chart redrawn from the report's efficiency table.

Riel Kediende Mapuor Akec

Electrical & Electronics Engineering graduate, Universiti Teknologi PETRONAS

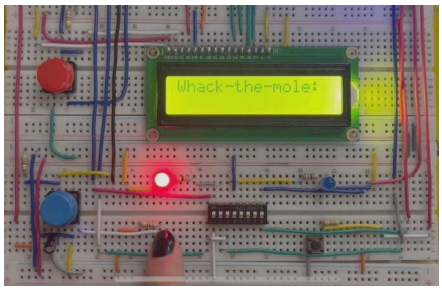
riel.akec@alum.utp.edu.my
linkedin.com/in/rielkediende

Whack-a-Mole Reaction Game on an STM32 Microcontroller

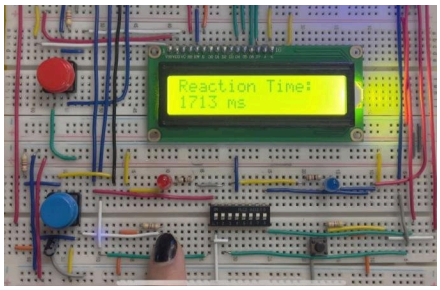
Microprocessor and Computer Architecture (EEB2083), UTP, August 2023 | Team of 2

A reaction-time game built on an STM32 Nucleo-F411RE (ARM Cortex-M4) and programmed in C with Mbed. After a random delay, a red or blue LED "mole" lights up and a hardware timer starts; pressing the matching button stops it and the 16×2 LCD shows the reaction time in milliseconds. A PWM buzzer gives an 800 Hz tone for a correct hit and 2 kHz for a wrong one, eight LEDs count the score in binary, and dedicated buttons pause, resume and reset the game.

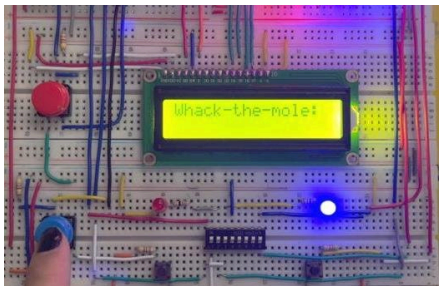
STM32F411RE	1 ms	800 Hz / 2 kHz	8-bit
ARM Cortex-M4, C with Mbed	reaction-time resolution (Timer)	PWM tones for right / wrong hit	LED score counter on a BusOut



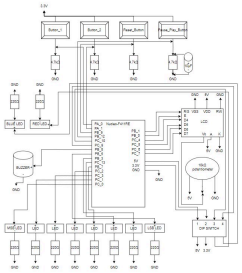
Mole up. Red LED lit and the LCD prompts the player to whack.



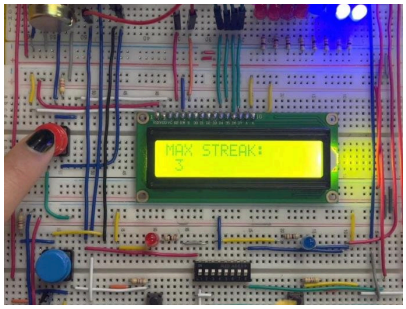
Measured reaction. Correct button pressed; LCD shows 1,713 ms.



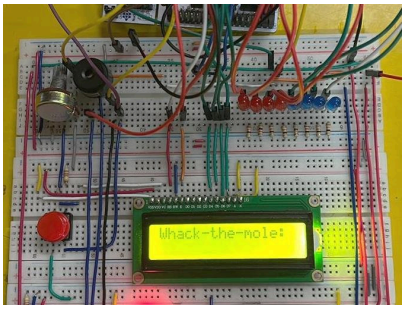
Paused. All eight score LEDs blink until play is pressed again.



Full schematic: Nucleo board, four buttons with pull-down resistors, mole LEDs, 8-LED counter, buzzer and LCD.



Game over. Reset shows the player's maximum streak.



Complete build on breadboard with the Nucleo-64.

What I did

- Wired the circuit and drew the schematic: GPIO inputs with pull-downs, LED outputs, PWM buzzer and the LCD in 4-bit mode.
- Wrote the game loop: random delay and mole selection, timer-based reaction measurement, and correct/wrong scoring.
- Added streak tracking, an 8-LED binary score display, and pause/resume and reset controls with button-release handling.
- Produced the flowchart and pseudocode, and debugged LED logic and LCD library issues.

```
// correct whack: score, streak, reaction time, PWM tone
if ((ledToWhack == 1 && button1Pin == 1) || (ledToWhack == 2 && button2Pin == 1)) {
  numCorrectlyWhacked++;  currentStreak++;
  if (currentStreak > maxStreak) maxStreak = currentStreak;
  ledpins = numCorrectlyWhacked;          // 8-LED binary counter (BusOut)
  lcd.printf("Reaction Time: ");
  lcd.locate(0, 1);
  lcd.printf("%d ms", reactionTime);      // from Timer t.read_ms()
  buzzer.period(1.0 / 800);               // 800 Hz tone
  buzzer.write(0.5);                      // 50% duty cycle
  wait_ms(100);
  buzzer.write(0.0);
}
```

Excerpt from the project code (lightly trimmed).

Photos, schematic and code from the team's project report, Microprocessor and Computer Architecture, Universiti Teknologi PETRONAS.

Riel Kediende Mapuor Akec

Electrical & Electronics Engineering graduate, Universiti Teknologi PETRONAS

riel.akec@alum.utp.edu.my
linkedin.com/in/rielkediende

12 V Linear DC Bench Power Supply with Adjustable Output

Team project (4 members), Electrical Circuit Analysis EFB1031, UTP, March 2023

A bench supply that steps 240 V mains down through a centre-tapped transformer, rectifies it with a two-diode full-wave rectifier, smooths it with a capacitor filter and regulates it into two outputs: a fixed 12 V rail from an L7812 and an adjustable rail from an LM317T. The adjustable rail drives a 12 V cooling fan for the regulators. We designed the circuit on paper, proved it on a breadboard, laid out the PCB in Eagle, then soldered it and built it into a ventilated acrylic enclosure with banana-plug terminals, a rocker switch and panel volt/ammeters.

240 V AC	12.68 V	1.25–13.75 V	14–30 V RMS
mains input via step-down transformer	fixed output measured (+5.7%)	adjustable output range (design)	safe transformer secondary window

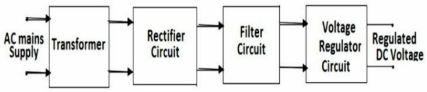
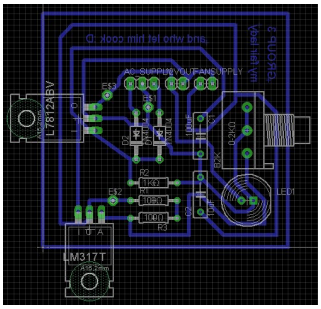
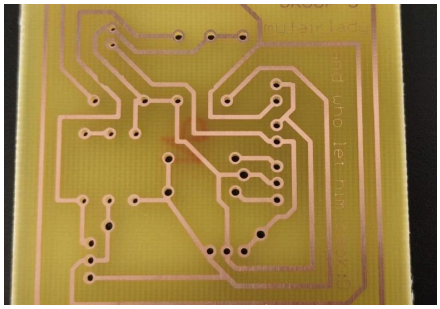


Figure 1. Block diagram of regulated DC power supply

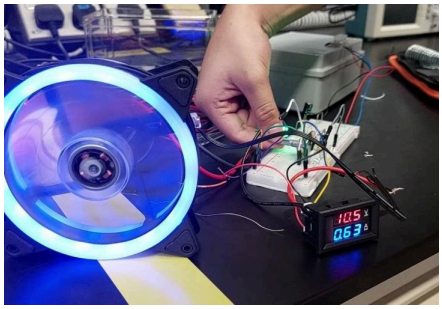
Signal chain. AC mains, transformer, rectifier, filter, voltage regulator, regulated DC output.



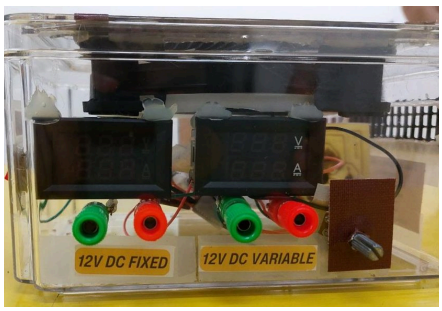
PCB layout in Eagle. L7812ABV and LM317T regulators, rectifier diodes, filter capacitors and LED indicator.



Fabricated PCB, matching the submitted Eagle design.



Breadboard prototype driving the 12 V fan, with a panel meter reading voltage and current.



Finished unit. Labelled 12 V fixed and 12 V variable outputs with panel meters.



Verification. Multimeter reading 12.69 V on the fixed output; oscilloscope used to check the filter output.

Design calculations

LM317 output

$V_0 = 1.25\ (1 + R2/R1)$

R2 = 2 kΩ, R1 = 200 Ω (2 × 100 Ω) gives 13.75 V max

Minimum input

$14\ V \times 1.414 = 19.8\ V$ peak, enough headroom above regulator dropout

Maximum input

$30\ V \times 1.414 = 42.4\ V$ peak, below the 50 V filter-capacitor rating

Problems we solved

- No output after assembly despite passing continuity: traced to the two regulators being swapped, then reworked.
- Lifted copper traces after desoldering: bridged with solder to restore the circuit.
- First enclosure failed to fit the PCB: rebuilt a second case around measured component dimensions, labelling and ventilation.
- Output 5.7% above nominal, attributed to heat-stressed joints and jumper resistance; noted for future builds.

Photos and measurements from the team's project report, Electrical Circuit Analysis (EFB1031), Universiti Teknologi PETRONAS.

Riel Kediende Mapuor Akec

Electrical & Electronics Engineering graduate, Universiti Teknologi PETRONAS

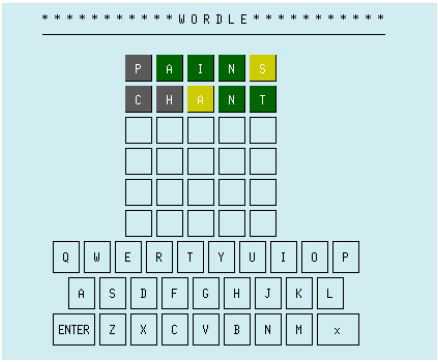
riel.akec@alum.utp.edu.my
linkedin.com/in/rielkediende

Wordle Game in C with a Mouse-Driven X11 Interface

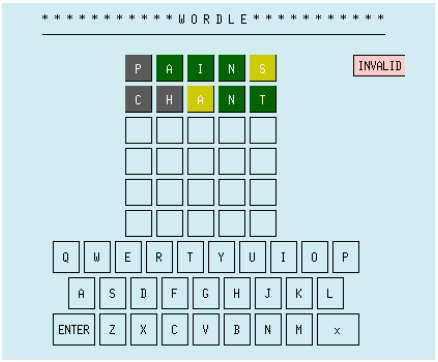
Structured Programming project, team of 2, UTP, September 2022 semester

A complete Wordle clone written in C on Linux. The player gets six tries to guess a hidden five-letter word using an on-screen keyboard; each guess is checked against the word list and coloured tile by tile. Everything on screen, from the menu and tile grid to the keyboard and feedback tiles, is drawn with a small X11 graphics library, and all input comes from mouse clicks mapped to on-screen coordinates.

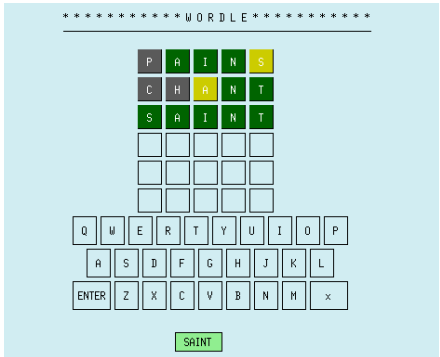
~680 lines	200 words	4 checks	0 repeats
of C, split into 6 game functions	loaded from a text file at runtime	dictionary, letters only, length, no repeats	target word never reused across games



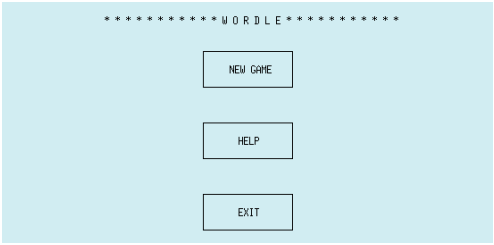
Feedback. Green = right letter, right place; yellow = in the word, wrong place; grey = not in the word.



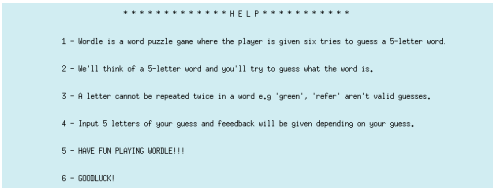
Input validation. "TRAIN" is not in the word list, so it is rejected with an INVALID flag and no guess is used up.



Win. Solved on the third guess; the answer is revealed below the board.



Main menu. New Game, Help and Exit buttons with click hit-testing.



Help screen with a Back button to the menu.

```c
// Valid only if the guess is in the 200-word list, letters only,
// exactly 5 characters and has no repeated letters
int validity(char* guess){
  ...
  while(in <= 200){                          // dictionary lookup
    fgets(tempo, WORD_SIZE, fp);
    if (strncmp(guess, tempo, NUM_OF_CHARS)==0){ invalid=0; break; }
    else invalid=1;  in++;
  }
  for(i=0;i<strlen(guess);i++)               // letters only
    if (isalpha(guess[i])==0) invalid=1;
  if (strlen(guess)!=NUM_OF_CHARS) invalid=1;  // length
  for(x=0;x<NUM_OF_CHARS;x++)                 // no repeats
    for(y=x+1;y<NUM_OF_CHARS;y++)
      if(guess[x]==guess[y]) invalid=1;
  return invalid;
}
```

Excerpt from the project code (lightly trimmed): the guess validation function.

What we built

- Event loop that turns mouse clicks into actions by hit-testing button and key coordinates stored in arrays.
- On-screen QWERTY keyboard with Enter and delete keys, writing letters into the tile grid as they are typed.
- Random word selection from the file, tracking used lines so no word repeats until all 200 are played.
- Letter-by-letter comparison producing green, yellow and grey feedback for each tile.
- In-game exit and end-of-game screens, returning cleanly to the main menu.
- Built with `gcc bankai.c gfx.c -lX11 -lm`; screenshots taken from a fresh build of the original code.

Screenshots captured on 26 Oct 2026 by compiling and playing the submitted source code. Structured Programming, Universiti Teknologi PETRONAS.